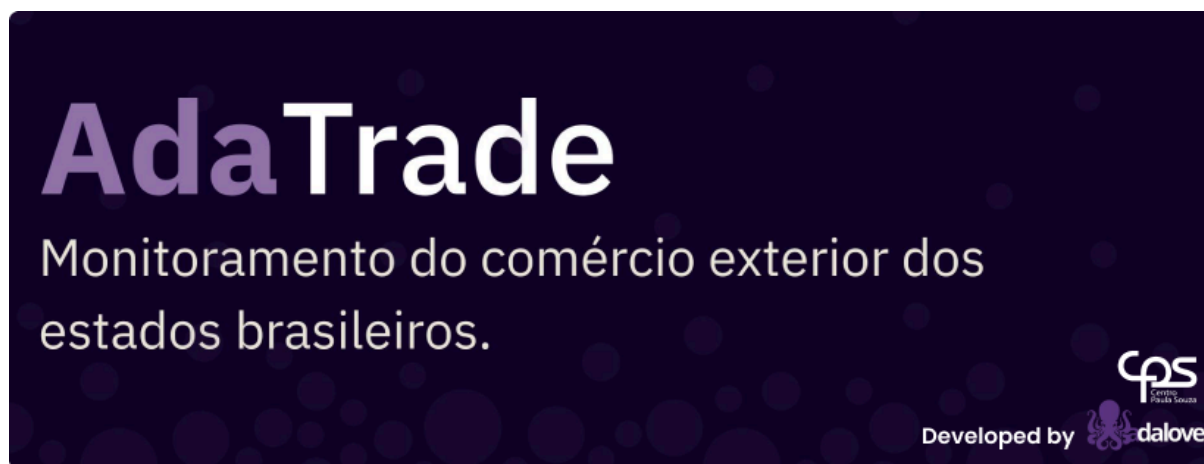


PROJETO API - SEMESTRE 3

Equipe Adalove



Descrição: Desenvolvimento de uma plataforma web que disponibilize informações sobre o desempenho dos Estados Brasileiros no comércio exterior, com base nos dados abertos do Ministério do Desenvolvimento, Indústria, Comércio e Serviços. Essa ferramenta fornecerá aos tomadores de decisão dados claros e acessíveis, permitindo a identificação de municípios que estejam em ascensão, estagnação ou declínio no mercado internacional.

[Acessar projeto no Github](#)

Sumário

1. Visão Geral -----	4
2. Requisitos -----	6
3. Arquitetura de Software -----	8
4. Design da Interface -----	14
5. Guia de Instalação e Configuração -----	
6. Histórico de Versões (Changelog) -----	
7. Manual do Usuário -----	
8. Referências e Glossário -----	

Sobre a Adalove

A organização foi fundada por estudantes da Faculdade de Tecnologia - Prof. Jessen Vidal, do curso Análise e Desenvolvimento de Sistemas

 Missão: Desbravar os horizontes da inovação tecnológica, oferecendo soluções eficazes que impulsionam o sucesso e a transformação digital. Com uma abordagem ágil e apaixonada, nossa equipe busca não apenas resolver problemas, mas também inspirar mudanças positivas.

 Visão: Ser reconhecida como uma líder inovadora no campo da tecnologia. Nossa visão vai além de simplesmente fornecer soluções tecnológicas; almejamos criar um ecossistema onde a inovação seja o motor do progresso. Enxergamos um futuro onde a Adalove é sinônimo de excelência em soluções tecnológicas, inspirando confiança e impulsionando o sucesso de nossos clientes em todos os setores.

 Valores: Inovação; Colaboração; Qualidade e Paixão!

Membro	Função	Github	Linkedin
Renan Tomasi	Master	Link do Perfil	Link do Perfil
Matheus Germano	Product Owner	Link do Perfil	Link do Perfil
Raphaela Barbosa	Desenvolvedor	Link do Perfil	Link do Perfil
Maria Fernanda Hansen	Desenvolvedor	Link do Perfil	Link do Perfil
Angelina Borroni	Desenvolvedor	Link do Perfil	Link do Perfil
Vitor Ribeiro	Desenvolvedor	Link do Perfil	Link do Perfil
Ramon	Desenvolvedor	Link do Perfil	Link do Perfil

1. Visão Geral

1.1. Aprendizagem Por Projeto Integrado (API)

Projeto pedagógico alicerçado na Metodologia API para ensino-aprendizado focado no desenvolvimento de competências e fundamentada nos pilares de aprendizado com problemas reais (RPBL), validação externa e mentalidade ágil.

Uso de estratégias para entender o problema, conceber uma solução viável ao desenvolver e implementar o MVP seguido de sua operação (CDIO).

Os resultados dos projetos devem obedecer ao Aviso Legal disponível no site da [Fatec SJC](#) com definição das datas do kickoff e das sprints.

1.2. Metodologia

O método Scrum é um método ágil adaptativo, iterativo, flexível e eficaz. Este método possui os seguintes princípios norteadores:

- Controle empírico
 - Auto-organização
 - Colaboração
 - Priorização da criação de valor
 - Time-boxing
 - Desenvolvimento iterativo
- Entre as ferramentas utilizadas no Scrum, uma é a divisão do projeto em Sprints.

Para selecionar quais seriam as entregas das nossas Sprints, primeiro definimos nosso MVP, priorizando as tarefas que trariam maior entrega de valor para o cliente. Então, a partir das Tarefas foi construído o Backlog do Produto, o qual foi aprovado pelo cliente e dividido em 4 Backlog de Sprint.

[Clique para visualizar os Backlogs](#)

1.3. Cronograma

Sprint	Data
Kick-Off	24/02 - 28/02
1° Sprint	10/03 - 30/03
Sprint Review / Planning	31/03 - 04/04
2° Sprint	07/04 - 27/04
Sprint Review / Planning	28/04 - 02/05
3° Sprint	05/05 - 25/05
Sprint Review	26/05 - 30/05
Feira de Soluções	17/06

MVP (Minimal Valuable Product)

AdaTrade é uma plataforma que permitirá que *economistas, gestores públicos e empresas* acompanhem e analisem o desempenho dos estados brasileiros no comércio exterior de maneira eficiente e acessível.

Descrição do Projeto: O AdaTrade é uma plataforma web que visa organizar e estruturar os dados de comércio exterior brasileiro, permitindo o acompanhamento detalhado do desempenho dos estados no mercado internacional. O sistema se baseia nos dados abertos do Ministério do Desenvolvimento, Indústria, Comércio e Serviços, oferecendo uma interface interativa para análise e tomada de decisão.

2. Requisitos

2.1. Requisitos Funcionais

- **Segmentação por Estado:**

A plataforma deverá permitir a navegação por unidades federativas, exibindo informações detalhadas de cada município, como produtos exportados/importados, volume de transações e variações históricas.

- **Busca e Filtros:**

O sistema deve oferecer funcionalidades de busca por código NCM (Nomenclatura Comum do Mercosul), além de filtros personalizáveis que possibilitem análises específicas por estado, período, tipo de carga, entre outros.

- **Integração com Dados Públicos do Governo:**

A plataforma deverá integrar-se com bases públicas oficiais, especialmente os dados fornecidos pelo Ministério do Desenvolvimento, Indústria, Comércio e Serviços (MDIC), garantindo atualização e confiabilidade das informações.

- **Exportação e Manipulação de Dados:**

Os usuários devem ter a opção de exportar os dados filtrados para formatos compatíveis com ferramentas como Power BI, possibilitando análises avançadas e construção de relatórios personalizados.

- **Análises Incorporadas (Dashboards):**

O sistema deverá apresentar dashboards interativos

2.3. Requisitos Não Funcionais

- **Frontend:**

A interface da aplicação será desenvolvida utilizando as seguintes tecnologias:

- **React.js:** biblioteca para construção de interfaces interativas.
- **Vite:** bundler moderno e rápido para desenvolvimento web.
- **TypeScript:** linguagem que adiciona tipagem estática ao JavaScript.
- **Tailwind CSS:** framework utilitário para estilização responsiva.
- **Shcn/UI:** biblioteca de componentes acessíveis e estilizados para React.

- **Backend:**

A lógica de negócios será implementada com o framework NestJS, que oferece uma arquitetura escalável, modular e baseada em boas práticas do Node.js.

- **Banco de Dados:**

O armazenamento dos dados será feito com o PostgreSQL,

- **Pipeline de Dados:**

A limpeza, normalização e transformação dos dados será realizada por meio de notebooks no Google Colab, utilizando Python e Pandas.

3. Arquitetura do Sistema

3.1. Arquitetura do Sistema

O sistema desenvolvido é uma aplicação web modular, composta por duas grandes camadas: **Frontend** e **Backend**, comunicando-se por meio de **requisições HTTP REST**. A aplicação tem como objetivo fornecer uma plataforma de visualização e análise de dados de comércio exterior brasileiro (exportações e importações).

3.1.1. Arquitetura Geral: Camadas

Camada de Apresentação (Front-end);

Camada de Aplicação (Back-end);

Camada de Persistência de Dados;

3.1.1.1. Camada de Apresentação (Front-end)

O frontend é uma **Single Page Application (SPA)** construída com **ReactJS**. Ele é responsável pela interface com o usuário, realizando:

- Consumo da API via **axios**.
- Renderização de gráficos e tabelas para exibição dos dados.
- Navegação dinâmica entre páginas usando **React Router DOM**.
- Sistema de filtros para consultas personalizadas baseadas em Estado, Ano e NCM.

O projeto é modularizado em componentes reutilizáveis, promovendo fácil manutenção e escalabilidade.

3.1.1.2. Camada de Aplicação (Back-end)

O backend é uma **API RESTful** desenvolvida em **NestJS**, aplicando conceitos de modularidade e injeção de dependência. Suas responsabilidades incluem:

- Gerenciamento dos dados de Exportações, Importações, NCMs, Estados e Municípios.
- Disponibilização de endpoints para consultas dinâmicas.
- Utilização do **Prisma ORM** para abstrair as operações no banco de dados.
- Implementação de DTOs para padronizar as requisições e respostas.

A arquitetura favorece a separação clara entre camadas de Controller, Service e Repository, aumentando a organização e a testabilidade do código.

3.1.1.3. Camada de Persistência de Dados

...

3.2. Fluxo do Sistema

O fluxo de funcionamento do sistema inicia-se no frontend, onde o usuário interage com a aplicação e define filtros ou parâmetros de busca, como ano, estado ou código NCM. A partir dessa interação, o frontend realiza chamadas HTTP, especificamente do tipo **GET**, para a API disponibilizada pelo backend. Essas requisições transportam os parâmetros definidos pelo usuário, solicitando os dados correspondentes.

Ao receber a requisição, o backend — desenvolvido em NestJS — interpreta os parâmetros e aciona os serviços internos responsáveis por gerenciar a lógica de acesso aos dados. Utilizando o **Prisma ORM**, o backend consulta o banco de dados relacional, aplicando os filtros recebidos para buscar exatamente as informações requeridas. Após a recuperação dos dados, o backend realiza um tratamento para formatá-los de maneira padronizada e os envia de volta ao front-end como resposta.

Com os dados recebidos, o frontend processa a resposta e organiza a apresentação das informações de forma visual e intuitiva para o usuário. Esses dados podem ser exibidos por meio de gráficos, tabelas ou outros componentes visuais, proporcionando uma análise clara e interativa dos dados de exportações e importações brasileiras.

3.3. Trade-Offs

Os trade-offs do projeto envolvem diversas decisões que afetam a qualidade, desempenho, custo e usabilidade do produto.

Separação Frontend e Backend

Decisão: Separar a aplicação em dois repositórios distintos, um para o frontend e outro para o backend, comunicando-se via API RESTful.

Trade-off:

- **Vantagem:** Facilita o desenvolvimento paralelo das equipes, melhora a organização do projeto e torna a manutenção futura mais simples, permitindo mudanças independentes em cada camada.
 - **Desvantagem:** Requer configuração adicional para integração e implantação em ambiente de produção, além de aumentar a complexidade do ambiente de desenvolvimento.
-

Simplicidade vs. Robustez na Comunicação

Decisão: Utilizar apenas requisições HTTP do tipo GET para buscar dados da API.

Trade-off:

- **Vantagem:** Simplifica o frontend e o backend, acelerando o desenvolvimento e reduzindo a curva de aprendizado dos envolvidos no projeto.
 - **Desvantagem:** Limita a expansão futura do sistema, já que a ausência de métodos como POST, PUT e DELETE impede operações de escrita e atualização, exigindo refatoração caso novas funcionalidades sejam adicionadas.
-

Segurança vs. Rapidez de Implementação

Decisão: Não implementar autenticação ou autorização nas requisições da API.

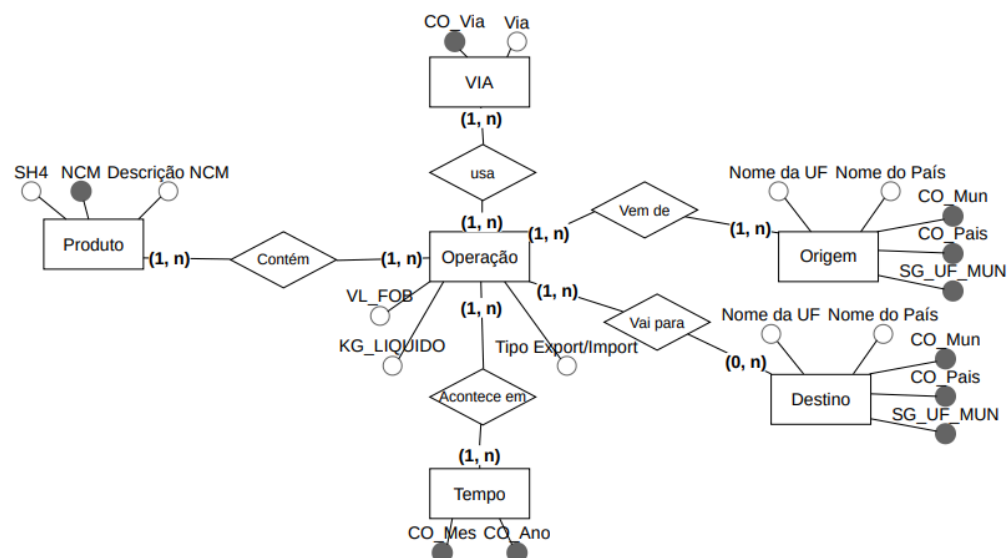
Trade-off:

Vantagem: Agiliza o desenvolvimento inicial e facilita o acesso livre aos dados para qualquer usuário.

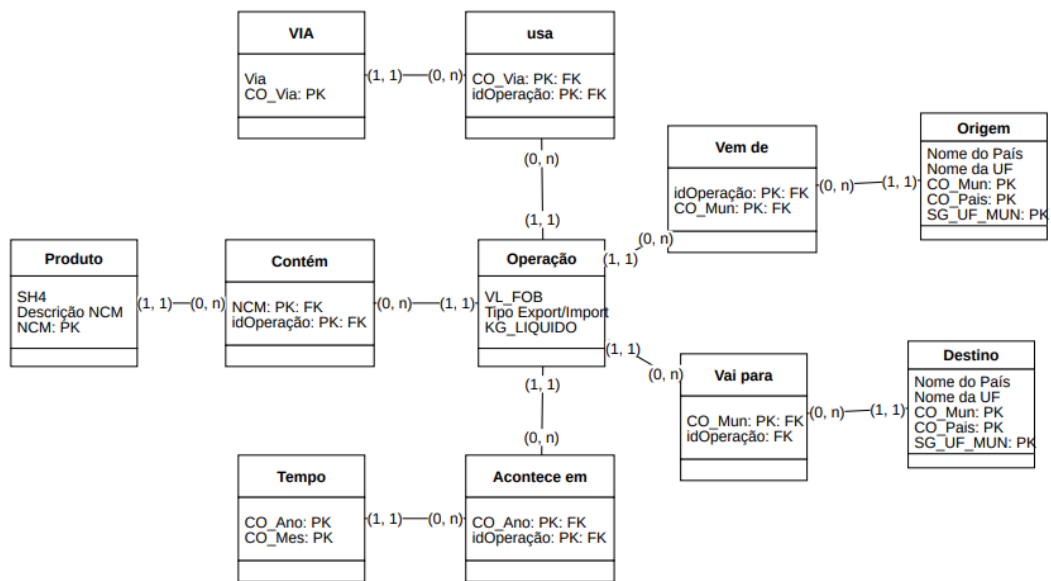
Desvantagem: Compromete a segurança da aplicação, expondo o backend a acessos indevidos e dificultando a proteção de dados em ambientes mais sensíveis ou críticos.

3.4. Diagramas

3.4.1. Modelo Conceitual



3.4.2. Modelo Conceitual



4. Design da interface

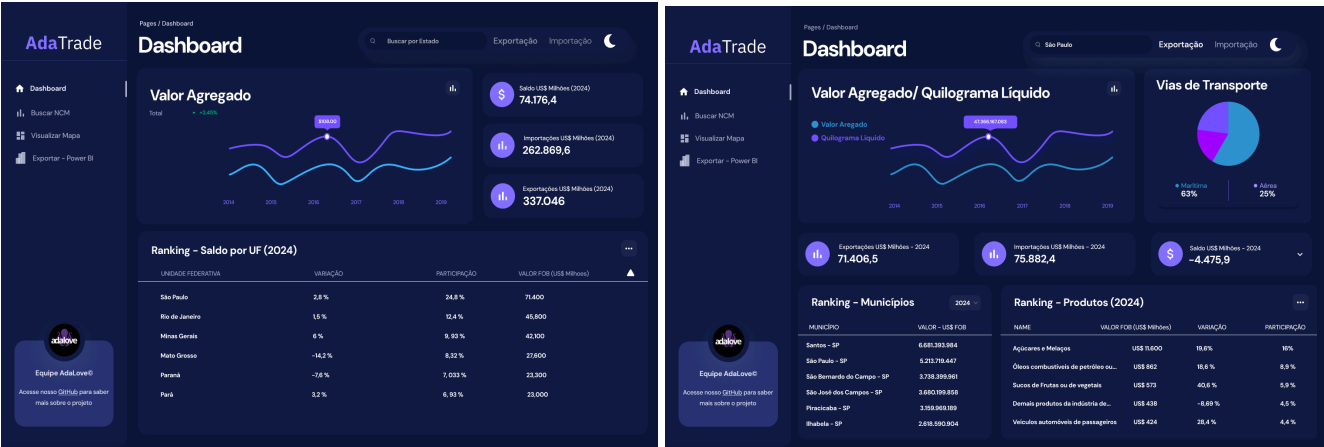
4.1. Design da Aplicação Web

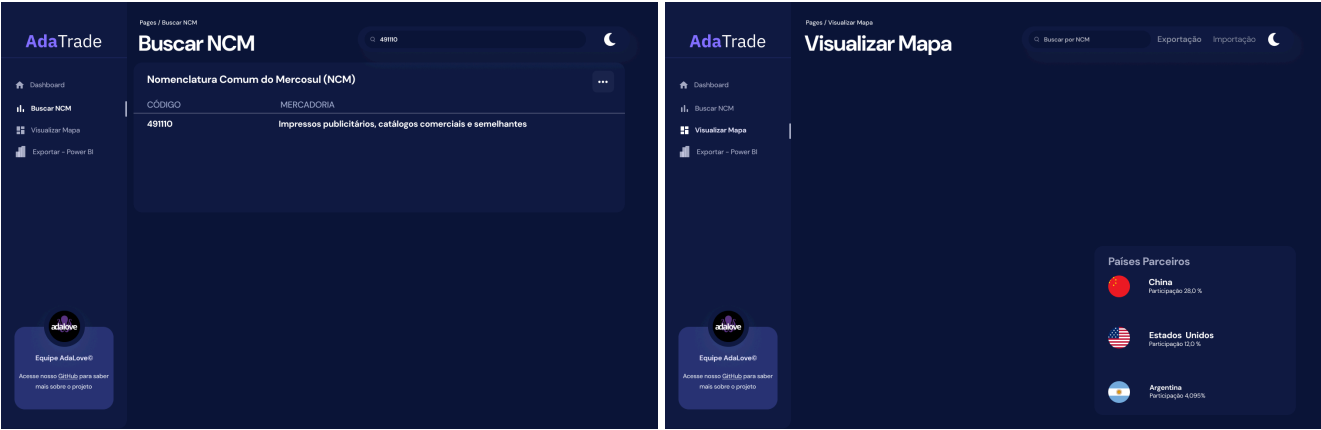
A aplicação contém uma interface minimalista e intuitiva, e como principal objetivo, melhorar a experiência do usuário final.

🎨 O time de design da equipe optou por uma paleta de cores em tons de roxo, visando o minimalismo e uma aplicação limpa com interface amigável ao usuário.

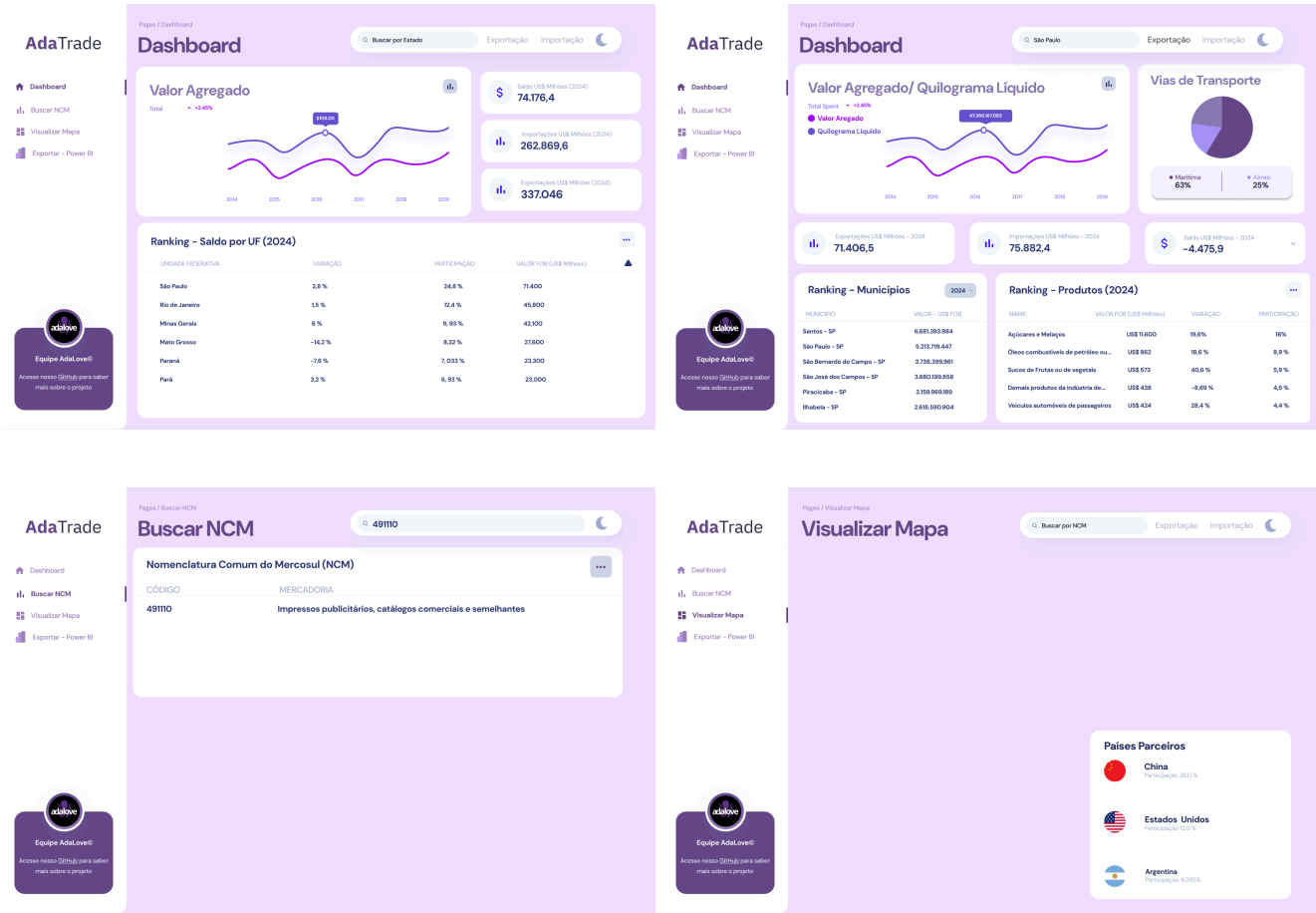
4.2. Protótipo

4.2.1 Modo Escuro





4.2.2 Modo Claro



5. Guia de Instalação e Configuração